

A Mechanically Verified Translation of $MTL_{0,\infty}$ into Timed Büchi Automata

Elie Fares, Jean-Paul Bodeveix, Hussain Al-Aqrabi, and Azar Salami



Abstract—We present a Coq-verified semantic translation from pointwise dense-time $MTL_{0,\infty}$ to event-guarded timed Büchi automata, without location invariants and with existential initial clock valuations. The translation first expresses ordinary timed operators using hatted Until and Release primitives, encodes their timing constraints in clocked LTL, and then uses an arbitrary LTL-to-Büchi backend satisfying an explicit semantic contract. A relaxation and reset-completion construction turns the backend automaton into a timed Büchi automaton. The central proof difficulty is reuse: one clock is assigned to each distinct primitive timed subformula, even when multiple activations of that obligation overlap. We establish soundness and completeness by a canonical reset policy and residual obligations, and prove the end-to-end theorem `MTL_to_TBA_correct_with`. The result preserves strict and non-strict bounds and does not depend on a particular LTL-to-Büchi implementation, provided it satisfies the stated contract. The intended use is as a verified semantic component in property-automaton construction for timed model checking; this artifact is not a standalone source-to-automaton compiler.

Index Terms—Metric temporal logic, timed Büchi automata, Coq, formal verification, clocked LTL, semantic preservation.

1 INTRODUCTION

Real-time requirements combine qualitative ordering with quantitative timing. Metric Temporal Logic (MTL) expresses both, while timed Büchi automata (TBA) provide a finite-state model suitable for timed verification. A translation between them is useful only when the automaton preserves the pointwise dense-time semantics of the formula. The translation itself is subtle: a timed subformula nested under an unbounded temporal operator may be activated under bound, and several instances of the same obligation may be pending at once.

A concrete use is dense-time verification of a bounded-response property such as $\Box(req \Rightarrow \Diamond_{\leq d} ack)$. If a second request arrives before an acknowledgment, two response obligations overlap. A property-automaton generator can use the theorem to justify sharing the subformula clock while retaining the older deadline; Table 2 traces two such requests and their common acknowledgment.

This paper gives a mechanized translation for the one-sided-bounded fragment $MTL_{0,\infty}$ [1]–[3]. Its key construction assigns a clock to each *distinct primitive timed subformula*,

rather than to each syntax-tree occurrence or run-time activation. The clocked-LTL encoding constrains that shared clock through reset and preservation markers. The proof’s completeness direction chooses a canonical reset trace that retains the oldest useful reference when it subsumes newer obligations; residual predicates express what remains of an obligation after time has advanced. This establishes that repeated activations need no additional clocks.

The translation is factored through ordinary LTL. It is therefore independent of the particular LTL-to-Büchi translator, subject only to a stated correctness contract on propositional words. After the backend, relaxation removes irrelevant negative extended-atom literals, and reset completion turns the relaxed cubes into timed transitions. The resulting theorem is mechanized in Coq for all eight hatted timed primitives, including strict bounds. The proof does not cover parsing, the interface to an external LTL translator, or output printing; these are outside the theorem. The Coq syntax stores bounds as elements of $\mathbb{R}$, so the result establishes semantic correctness for Coq-level formulas; it does not provide a finite textual encoding or an extracted executable compiler for numeric bounds.

Our contributions are:

- a pointwise-semantics-preserving clocked-LTL encoding of the eight hatted Until and Release primitives, with ordinary timed operators derived by semantic identities;
- soundness and completeness of that encoding, including a canonical reset policy and residual obligations that justify one clock per distinct primitive timed subformula despite arbitrarily overlapping activations, without dropping older pending deadlines;
- relaxation and reset completion, with an end-to-end Coq theorem for every LTL-to-Büchi backend satisfying the explicit correctness contract.

The formal development includes the named results `EncodingCorrect_proved`, `T_uses_every_clock`, and `MTL_to_TBA_correct_with`. The complete arguments for the canonical policy and its strict-bound cases appear in the supplementary material. This paper treats the translation through TBA construction; generic TBA optimization and UPPAAL export are separate contributions and are not needed for the theorem proved here.

The theorem supplies a proof-backed semantic component for a verification pipeline whose LTL-to-Büchi backend

Elie Fares, Hussain Al-Aqrabi, and Azar Salami are with the Higher Colleges of Technology, Sharjah, United Arab Emirates (e-mail: efares@hct.ac.ae; halaqrabi@hct.ac.ae; asalami@hct.ac.ae).

Jean-Paul Bodeveix is with IRIT, Université de Toulouse, France (e-mail: bodeveix@irit.fr).

satisfies the stated contract. The included artifact is not a drop-in translator: parsing, finite numeric-bound encoding, backend invocation, zero-initialization adaptation, and automaton export remain outside this result.

The Spot 2.16 example in Fig. 4 documents the propositional-backend interface, but the Coq proof does not establish Spot's correctness contract.

The broad translation capability is not new: CASAAL already translates $\text{MTL}_{0,\infty}$ formulas to timed automata. Our claim is the machine-checked soundness and completeness argument for this clocked-LTL factorization, in particular the formula-keyed clock policy under overlapping activations. We do not claim the first translation or an automaton-size advantage.

2 LOGIC AND SEMANTIC MODEL

2.1 Pointwise $\text{MTL}_{0,\infty}$

Let Σ be a finite set of actions. Formulas are in negation normal form and use Boolean operators, next, untimed Until and Release, and timed Until and Release whose intervals have one endpoint at zero or infinity. We write a bound as $\sim d$, with $\sim \in \{<, \leq, >, \geq\}$. The logic also contains the hatted operators $\hat{\text{U}}_{\sim d}$ and $\hat{\text{R}}_{\sim d}$. In a hatted operator the current position is skipped by the logical interval, while its timestamp remains the timing reference.

A timed word is $w = (a_i, t_i)_{i \in \mathbb{N}}$, where $a_i \in \Sigma$, $0 \leq t_0 < t_1 < \dots$, and timestamps diverge. We write $\text{elap}_w(i, j) = t_j - t_i$. For Until, $w, i \models_{\text{MTL}} p \text{U}_{\sim d} q$ holds when some $j \geq i$ satisfies q and $t_j - t_i \sim d$, with p at each position in $[i, j)$. For hatted Until, the witness is $j > i$ and p is required on (i, j) . Release is the dual universal condition: at every position in the bounded region, either q holds or p held earlier in the corresponding interval. Hatted Release excludes the current position from that region. We use the standard pointwise semantics [4].

For precision, the four bounded clauses used below are:

$$\begin{aligned}
 w, i \models_{\text{MTL}} p \text{U}_{\sim d} q &\iff \exists j \geq i. w, j \models_{\text{MTL}} q \\
 &\quad \wedge t_j - t_i \sim d \\
 &\quad \wedge \forall k \in [i, j). w, k \models_{\text{MTL}} p, \\
 w, i \models_{\text{MTL}} p \hat{\text{U}}_{\sim d} q &\iff \exists j > i. w, j \models_{\text{MTL}} q \\
 &\quad \wedge t_j - t_i \sim d \\
 &\quad \wedge \forall k \in (i, j). w, k \models_{\text{MTL}} p, \\
 w, i \models_{\text{MTL}} p \text{R}_{\sim d} q &\iff \forall j \geq i. \\
 &\quad \left(t_j - t_i \sim d \Rightarrow \right. \\
 &\quad \left. \left(w, j \models_{\text{MTL}} q \right. \right. \\
 &\quad \left. \left. \vee \exists k \in [i, j). w, k \models_{\text{MTL}} p \right) \right), \\
 w, i \models_{\text{MTL}} p \hat{\text{R}}_{\sim d} q &\iff \forall j > i. \\
 &\quad \left(t_j - t_i \sim d \Rightarrow \right. \\
 &\quad \left. \left(w, j \models_{\text{MTL}} q \right. \right. \\
 &\quad \left. \left. \vee \exists k \in (i, j). w, k \models_{\text{MTL}} p \right) \right).
 \end{aligned}$$

The index intervals refer to event positions, whereas $t_j - t_i$ is physical time. This distinction is material when no event occurs exactly at a bound. The strict increase assumption

excludes simultaneous events; divergence is used in the lower-bounded Until fairness argument.

The Coq development represents action names by natural numbers rather than a parameterized finite type, and timed words may use any natural-number name. Each formula mentions finitely many names, so its semantics and generated labels distinguish actions only by equality or inequality with those names. Equivalently, the construction operates over the finite quotient induced by those action tests; this is the finite alphabet used in the mathematical presentation.

The well-formedness condition requires $d > 0$ for every lower bound and strict upper bound, and $d \geq 0$ for a non-strict upper bound. Ordinary timed operators are definitions in the mechanized core. For upper bounds $\preceq \in \{\leq, <\}$ and lower bounds $\succeq \in \{\geq, >\}$, they are:

$$p \text{U}_{\preceq d} q \equiv q \vee (p \wedge (p \hat{\text{U}}_{\preceq d} q)), \quad (1)$$

$$p \text{U}_{\succeq d} q \equiv p \wedge (p \hat{\text{U}}_{\succeq d} q), \quad (2)$$

$$p \text{R}_{\preceq d} q \equiv q \wedge (p \vee (p \hat{\text{R}}_{\preceq d} q)), \quad (3)$$

$$p \text{R}_{\succeq d} q \equiv p \vee (p \hat{\text{R}}_{\succeq d} q). \quad (4)$$

These definitions separate the current-position case from the strictly future obligation. Their pointwise semantic compatibility, including the stated bound hypotheses, is proved in Coq:

Theorem 2.1 (Semantic compatibility of the derived timed operators). *Under the well-formedness conditions above, Eqs. (1)–(4) agree with the standard pointwise semantics for all strict and non-strict upper and lower bounds.*

The identities follow by separating the current-position case from a strictly future endpoint. For upper Until, a witness at i contributes q ; every later witness requires p at i and leaves the hatted obligation. A lower bound with $d > 0$ excludes i , so p is required now and the remaining witness is hatted. The Release identities are the dual case split: an upper-bound region includes i and therefore requires q there, while a lower-bound region excludes i . Strict upper bounds also require $d > 0$, avoiding the degenerate < 0 case. The eight corresponding Coq lemmas are listed in Supplement 2.2. Thus only the eight hatted timed constructors require primitive translation clauses.

This distinction is useful for sporadicity: if consecutive occurrences of e must be separated by at least θ , one may write $\Box(e \Rightarrow \hat{\Box}_{<\theta} \neg e)$. The hatted form retains the triggering event as the time origin; ordinary bounded temporal operators do not express this particular reference shift directly. Related pointwise MTL constructions and the CASAAL translation are discussed in prior work [5]–[8].

2.2 Timed Büchi Automata

The verified target is an event-guarded TBA $\mathcal{A} = (L, \ell_0, X, E, \text{Acc})$. Here L is a finite set of locations, ℓ_0 the initial location, X a finite set of clocks, $E \subseteq L \times \text{Lab} \times \text{Guard} \times 2^X \times L$ is the set of action-labelled transitions with guards and reset sets, and $\text{Acc} \subseteq L$ the Büchi set. A run is a sequence $(\ell_i, v_i)_{i \geq 0}$ starting at ℓ_0 ; it follows one transition per event and visits Acc infinitely often. If the selected edge at event i is $(\ell_i, \alpha_i, g_i, Z_i, \ell_{i+1})$, then the event action must

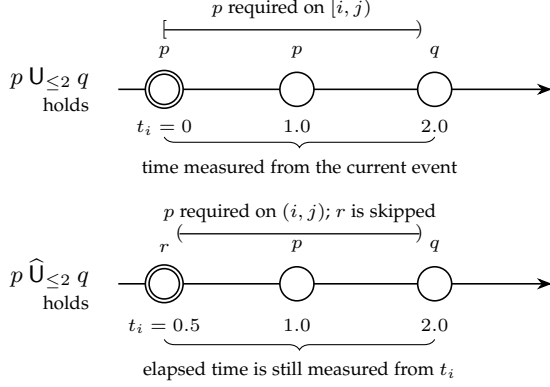


Fig. 1. Ordinary and hatted Until. The double circle is the timing reference. The hatted operator skips the current event logically but measures from its timestamp.

satisfy α_i and the valuation v_i must satisfy every guard in g_i . There are no location invariants in this model: timing constraints are checked at the events of the timed word. This is the TBA record represented in Coq; the theorem does not cover a richer invariant-bearing automaton model without an additional encoding.

Let v_i be the clock valuation tested at event i and $\Delta_i = t_{i+1} - t_i$. A transition at event i tests v_i before its reset decision; the next valuation is

$$v_{i+1}(x) = \begin{cases} \Delta_i, & x \in Z_i, \\ v_i(x) + \Delta_i, & \text{otherwise.} \end{cases}$$

The initial nonnegative valuation v_0 is existentially quantified, as in the Coq clock-consistent-extension semantics. In the example below, the first transition resets the deadline clock at the origin event, and every outgoing transition from the waiting location checks the upper deadline. The standard zero-initialization convention is not assumed by the theorem.

3 CLOCKED-LTL TRANSLATION

3.1 Shared clock identities

The translation is structural. It maps Boolean and untimed temporal operators homomorphically and maps each primitive timed subformula to an LTL formula over an extended alphabet. For each action a , the extended alphabet contains distinct atoms $\text{LAct}(a)$ and $\text{LNAct}(a)$, as well as clock comparisons $x \sim d$, a reset atom $\text{rst}(x)$, and a preservation atom $\text{unch}(x)$. Under timed-word interpretation, the action atoms mean that the current action is a or is different from a , respectively. The atom $\text{LNAct}(a)$ and the negated literal of $\text{LAct}(a)$ have the same timed-word interpretation, but remain distinct propositions to the unconstrained LTL backend. The reset and preservation atoms describe the clock update at the current event. Clock values are read before the event's reset, matching the guard semantics of a timed automaton.

For formula f , let $\text{Clock}(f)$ be its finite set of distinct primitive hatted timed subformulas. The clock associated with $F \in \text{Clock}(f)$ is $x = F$ itself. Thus two identical timed subformulas at different syntax-tree paths denote the same

logical clock. Syntactically different subformulas remain distinct in this construction. The main result proves that the clock also serves all dynamic activations of that subformula. The clock count is therefore bounded by the number of distinct primitive timed subformulas, which is at most the number of timed occurrences in f . If structural subformulas are represented as shared nodes, the translation can share their LTL encodings as well; this does not remove the usual worst-case growth of an LTL-to-Büchi construction.

This sharing is semantic rather than an automaton-state minimization. The clock key is the normalized primitive formula, including its comparison and constant, so identical obligations have one clock identity before the propositional automaton is built. The reset trace is then chosen for that identity as a whole. In particular, the proof cannot justify the clock bound by assigning a fresh clock to each activation and subsequently identifying those clocks: it establishes directly that a single trace of reset decisions for the formula is sufficient.

3.2 Eight primitive clauses

Let F be a primitive hatted formula at path π , with operands p, q . Write $x = \text{clock_of}(F)$, $P = \tau_{\pi 0}(p)$, $Q = \tau_{\pi 1}(q)$, and define $\alpha W \beta = (\alpha U \beta) \vee G\alpha$. For upper comparisons $\lesssim \in \{\leq, <\}$ and lower comparisons $\gtrsim \in \{\geq, >\}$, complements are $\leq^c = >$, $<^c = \geq$, $\geq^c = <$, and $>^c = \leq$. The four clause shapes, each instantiated for its strict and non-strict bound, give all eight primitive cases:

$$\tau_{\pi}(p \hat{U}_{\lesssim d} q) = X(((x \lesssim d) \wedge \text{unch}(x) \wedge P) \vee ((x \lesssim d) \wedge Q)), \quad (5)$$

$$\tau_{\pi}(p \hat{U}_{\gtrsim d} q) = \text{rst}(x) \wedge X((P U ((x \gtrsim d) \wedge (P U Q))) \vee (GP \wedge GFQ)), \quad (6)$$

$$\tau_{\pi}(p \hat{R}_{\lesssim d} q) = \text{rst}(x) \wedge X(Q \vee ((x \lesssim^c d) \vee (P \wedge Q))), \quad (7)$$

$$\tau_{\pi}(p \hat{R}_{\gtrsim d} q) = X(((x \gtrsim^c d) \wedge \text{unch}(x)) \vee ((P R Q) \vee ((x \gtrsim^c d) \wedge P))). \quad (8)$$

All clock atoms occur positively. Upper-bounded hatted Until uses strong Until, matching the mechanized clause: its witness must satisfy q while the clock remains within the upper bound. On a clock-consistent trace over divergent time, an infinite continuation cannot keep the preservation guard true forever. Lower-bounded Until retains its fairness branch, including GFQ , so that eventuality is not lost. The weak-Until clauses mark the end of bounded Release windows. Strict comparisons are represented directly; no strict bound is weakened by adjusting its constant.

The clauses have four distinct temporal roles. For upper-bounded Until, the clock is tested before any new reset: while its value remains in the allowed window, P must continue until Q appears. If the bound is passed first, no bounded witness exists and the strong-Until clause fails. For lower-bounded Until, a finite path waits for the threshold and then starts the unbounded $P U Q$ obligation. If the threshold is never reached, the second disjunct requires P at every position and Q infinitely often. Time divergence is

TABLE 1
Clock comparisons used by each hatted primitive.

Primitive role	Non-strict bound	Strict bound
$\widehat{U}_{\lesssim d}$ endpoint	$x \leq d$	$x < d$
$\widehat{U}_{\gtrsim d}$ threshold	$x \geq d$	$x > d$
$\widehat{R}_{\lesssim d}$ window end	$x > d$	$x \geq d$
$\widehat{R}_{\gtrsim d}$ pre-threshold	$x < d$	$x \leq d$

what makes this infinite branch equivalent to a witness at distance at least d .

For upper-bounded Release, the initial Q is required and the weak-until stops either at the complementary clock comparison or when $P \wedge Q$ discharges the obligation. For lower-bounded Release, the clock remains unchanged before the threshold; after that point, the translated obligation is ordinary Release, with the endpoint alternative covering the transition into the threshold region. Thus Until clauses use existential witnesses, whereas Release clauses maintain a universal condition over the bounded region. The hatted shift is encoded by the outer X in every clause; for restart families, the separate $\text{rst}(x)$ atom denotes the reference timestamp at the current position.

3.3 Why hatted operators and shared clocks matter

The hatted operators preserve the triggering timestamp while moving logical evaluation to the next event. The same distinction appears in the clocked encoding: the initial X moves the temporal obligation forward, while the clock retains the reference selected by its reset trace.

For instance, $\Box(e \Rightarrow \widehat{\Box}_{<\theta} \neg e)$ measures the exclusion window from the triggering event. Prefixing the ordinary operator with $\bigcirc$ shifts the reference to the next event, while omitting $\bigcirc$ also constrains the triggering event itself. The three readings differ on the same timed word, as shown in Fig. 2.

Clock sharing has two layers. First, identical primitive formulas at different syntax-tree paths share one clock by construction. Second, that single clock must represent all pending activations. For example, in $\Box(\text{req} \Rightarrow \Diamond_{\leq 5} \text{ack})$, requests at times 0 and 2 can both be pending before an acknowledgement. Retaining the reference from time 0 suffices: an acknowledgement at time 4 is within five time units of both requests. Table 2 traces this case. Clock values are measured before the reset decision at each event; at the acknowledgement, the guard sees age four and the successful endpoint discharges both activations. For any newer request at time t_k between an older activation t_i and a witness t_j , $t_j - t_k \leq t_j - t_i$; the same endpoint therefore satisfies the newer upper-bound obligation. The argument depends on the required P condition holding between activations and the endpoint, which is exactly what the Until residual records.

For the other primitive families, the canonical policy and residual predicates handle the direction in which restarting or preserving the clock is required (Supplement 2.6).

TABLE 2
One shared upper-Until clock for overlapping requests.

Event i	Action	Time t_i	Clock age before decision
0	<i>req</i>	0	0
1	<i>req</i>	2	2
2	<i>ack</i>	4	4

3.4 Canonical reset policy

The encoding quantifies over clock traces. To prove completeness with one clock for a formula that may be activated arbitrarily often, it is enough to construct one trace that gives priority to an older activation only while that activation remains useful. The proof records usefulness by residual obligations. For upper-bounded Until, the residual says that a witness is still reachable from the current clock age:

$$\begin{aligned} \text{URes}_{\lesssim}(i, v, d, p, q) \iff \exists j \geq i. v + \text{elap}_w(i, j) \lesssim d \\ \wedge w, j \models_{\text{MTL}} q \\ \wedge \forall k \in [i, j). \\ w, k \models_{\text{MTL}} p, \end{aligned}$$

where $\lesssim$ is either $\leq$ or $<$. For lower-bounded Release, the residual records that every position beyond the threshold is still covered:

$$\begin{aligned} \text{RRes}_{\geq}(i, v, d, p, q) \iff \forall j \geq i. d \leq v + \text{elap}_w(i, j) \Rightarrow \\ \left(w, j \models_{\text{MTL}} q \right. \\ \left. \vee \exists k \in [i, j). \right. \\ \left. w, k \models_{\text{MTL}} p \right), \\ \text{RRes}_{>}(i, v, d, p, q) \iff \forall j \geq i. d < v + \text{elap}_w(i, j) \Rightarrow \\ \left(w, j \models_{\text{MTL}} q \right. \\ \left. \vee \exists k \in [i, j). \right. \\ \left. w, k \models_{\text{MTL}} p \right). \end{aligned}$$

The strict predicates preserve the strict comparison itself. They are defined and shifted in the Coq development (the full definitions and one-step lemmas are in Supplement 2.6).

The residual predicates are deliberately asymmetric. An upper-Until residual is existential: one still-reachable endpoint is enough, and that endpoint can discharge the obligation initiated at the current position. A lower-Release residual is universal: every position that has entered the threshold region must satisfy the Release condition. The canonical policy preserves a clock only when its residual remains true, so clock reuse follows from the semantic shape of the obligation rather than a syntactic rule such as “reset on every activation.” This distinction is needed for repeated activations, but the policy is not part of the generated automaton: it is a word-dependent witness used only to prove completeness.

For clock $x = F$ with current value v , the canonical policy is summarized in Table 3. “Preserve iff” means that the clock is not reset exactly under the listed condition; in the remaining cases it is reset. For the restart families, the stated test is the reset condition. The canonical extension

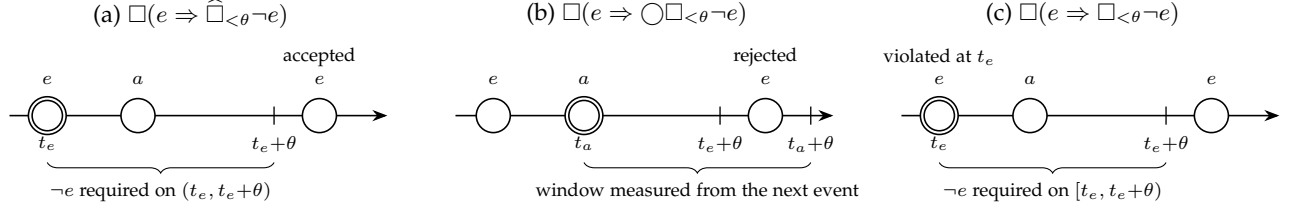


Fig. 2. The hatted sporadicity property and two ordinary-operator substitutions. A leading next operator shifts the timing reference to the following event; omitting it includes the triggering event in the window.

TABLE 3
Canonical reset policy for each timed-operator family.

Primitive clock F	Reset decision at position i
$p \hat{U}_{\leq d} q$	Preserve iff $v \leq d$, $\text{URes}_{\leq}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} q$.
$p \hat{U}_{< d} q$	Preserve iff $v < d$, $\text{URes}_{<}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} q$.
$p \hat{U}_{\geq d} q, p \hat{U}_{> d} q, p \hat{R}_{\leq d} q,$ $p \hat{R}_{< d} q$	Reset iff $w, i \models_{\text{MTL}} F$.
$p \hat{R}_{\geq d} q$	Preserve iff $v < d$, $\text{RRes}_{\geq}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} p$.
$p \hat{R}_{> d} q$	Preserve iff $v \leq d$, $\text{RRes}_{>}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} p$.

uses this decision for the formula-keyed clock and updates its value by

$$u_0 = 0, \quad u_{i+1} = \begin{cases} \Delta_i, & \text{if the policy resets at } i, \\ u_i + \Delta_i, & \text{otherwise.} \end{cases}$$

The policy depends on truth in the timed word and, through residuals, on its future. It is a semantic witness used in the completeness proof, not an algorithm executed by the translator. Its input is the timed formula itself, not a syntax-tree path, so identical occurrences share the same reset trace.

The two preservation cases retain an older reference only if the residual is still true and the current event has not discharged it. A later endpoint that satisfies the older upper-bounded Until also satisfies newer pending copies with shorter remaining prefixes. For lower-bounded Release, an old reference reaches the threshold no later than a newer one; the residual shift $(i, v) \mapsto (i + 1, v + \Delta_i)$ carries the remaining obligation. Conversely, lower-bounded Until and upper-bounded Release restart when the current formula is true; their translated continuation keeps any older obligations represented. The strict cases follow the same organization with strict residuals and guards.

3.5 Why one trace suffices for shared activations

The clock key is the primitive timed formula, not its syntax-tree path or the dynamic activation that created an obligation. Thus identical subformulas at different tree positions use the same value and reset trace; the Coq lemma `identical_nodes_share_canonical_trace` proves this equality. Dynamic activations still impose distinct semantic obligations. The canonical policy represents them by retaining an older clock reference only while its residual obligation remains valid.

For an upper-bounded Until, let activations at $i_0 \leq i_1 < j$ share an endpoint j . The older reference gives the stronger deadline constraint,

$$t_j - t_{i_1} \leq t_j - t_{i_0} \leq d.$$

Its required p -prefix contains the suffix required by the newer activation. Consequently, preserving the older clock while $\text{URes}_{\leq}$ holds covers both activations; when the older residual fails or its endpoint has discharged it, the policy can reset for a new obligation. This is the invariant used by `complete_UhatLe` and its strict counterpart.

Lower-bounded Release has the dual universal shape. An older reference has at least the age of a newer one and therefore reaches the lower threshold no later. The predicate RRes records whether every position beyond that threshold remains covered by q or an earlier p . While the clock is preserved, the residual advances by

$$(i, v) \mapsto (i + 1, v + \Delta_i),$$

which is the one-step lemma `rge_residual_shift` used by `complete_RhatGe`. In the dual restart families, a current activation resets the clock, while the Γ branch for lower-bounded Until and weak-until continuation for upper-bounded Release retain the older obligations in the LTL formula. The same cases use strict residuals and strict guards for the strict primitives. The resulting sharing argument is about semantic obligations, not an assumption that activations are nonoverlapping.

4 FROM CLOCKED LTL TO A TBA

4.1 LTL-to-Büchi contract

The backend treats extended atoms as independent propositions. It returns a propositional Büchi automaton whose language is exactly that of the clocked-LTL formula. The theorem is stated for any such backend, deterministic or not.

Assumption 4.1 (Backend correctness). *For each formula F and each propositional word s over its atoms, $\text{PBA_accepts}(A, s)$ iff $s, 0 \models F$, where A is the backend automaton for F .*

This contract ranges over all propositional words, including assignments that do not correspond to any clock-consistent timed word. That stronger scope is used by relaxation.

4.2 Relaxation and reset completion

The clocked-LTL formula has positive occurrences of extended atoms. Its propositional language is consequently

upward closed in those atoms. In each backend cube, relaxation deletes negative literals over extended atoms and irrelevant extended atoms; contradictory cubes are discarded. This preserves the accepted language because any satisfying assignment can be adjusted to the original cube while remaining a model of the positive formula.

Each clock belongs to one of two classes. A preservation clock is used with upper-bound tests and $\text{unch}(x)$, and a restart clock is used with lower-bound tests and $\text{rst}(x)$. Reset completion converts a relaxed cube into a TBA transition as follows:

- reset a preservation clock unless the cube contains $\text{unch}(x)$;
- reset a restart clock exactly when the cube contains $\text{rst}(x)$;
- remove reset and preservation markers, retaining clock tests as the transition guard and action literals as its label.

The asymmetry is essential. Additional resets only decrease preservation clock values and hence preserve upper tests. A restart clock can be reset only when explicitly required, since decreasing it could invalidate a lower test. The full monotonicity argument is given in Supplement 1.1.

Lemma 4.2 (Reset completion). *For every accepting run of the relaxed backend automaton on a clock-consistent extension, there is a clock-consistent extension with the same timed word and initial values whose resets are exactly those prescribed by reset completion, and the same run is accepting in the resulting TBA.*

Proposition 4.3 (Exactness of relaxation and reset completion). *Let f be well formed and let A satisfy Assumption 4.1 for $\mathcal{T}(f)$. The TBA obtained from A by relaxation and reset completion accepts exactly the timed words w for which $w, 0 \models_{\text{MTL}} f$, under existential initial clock values.*

The relaxation proof uses positivity of the clock atoms in $\mathcal{T}(f)$. A run of the original Büchi automaton remains a run after relaxation. In the reverse direction, take a run of the relaxed automaton and adjust each propositional assignment only on the extended atoms whose negative literals were deleted, making those atoms false; set irrelevant deleted atoms to true where needed to recover the original cube. The backend contract makes the adjusted word a model of $\mathcal{T}(f)$. On atoms that occur in $\mathcal{T}(f)$, the relaxed word only adds truth, so upward closure makes it a model too. Reset completion then assigns resets monotonically: preservation clocks become no larger, keeping upper guards true, and restart clocks become no smaller, keeping lower guards true. Thus the same Büchi run remains accepting. The proof is mechanized as `relax_sound`, `relax_complete`, and `complete_complete`.

Acceptance convention. The theorem quantifies existentially over the nonnegative clock values at the first event. This is a property of the accepted timed language; it does not by itself establish equivalence with an automaton whose clocks must start at zero. For example, a first transition guarded by $x \geq 1$ may be enabled for one existential initial value and disabled when the first event is at time zero. The worked example avoids dependence on that value by resetting its deadline clock on the first transition. The

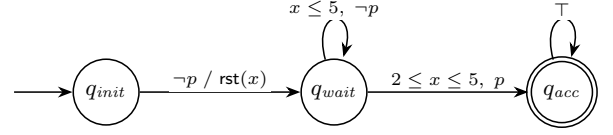


Fig. 3. An equivalent event-guarded TBA for f . The initial transition requires $\neg p$ and resets x . In the waiting location, every outgoing transition checks $x \leq 5$; the transition to the accepting location also requires $2 \leq x$. Thus p can first hold only at an event whose age from the origin is in $[2, 5]$.

conventional initialization question is otherwise outside Theorem 5.3.

4.3 Worked example: translating a bounded response

Consider

$$f = (F_{\leq 5} p) \wedge (G_{< 2} \neg p).$$

The property requires p to be false at the temporal origin and at every later event less than two time units from it, and requires a p event by age five. Figure 3 shows an equivalent, one-clock event-guarded TBA. Its first transition consumes the event at the origin, requires $\neg p$, and resets x . Under the event-indexed reset convention of Section 2.2, the clock value at the next event is therefore the elapsed time since the origin. The construction below follows the formula through the clocked-LTL intermediate language and the timed reinterpretation.

Step 1: The formula is in negation normal form. Expanding the derived operators and applying the timed-operator identities gives

$$\begin{aligned} f &= (\top \cup_{\leq 5} p) \wedge (\perp R_{< 2} \neg p) \\ &= (p \vee (\top \wedge \top \hat{\cup}_{\leq 5} p)) \\ &\quad \wedge (\neg p \wedge (\perp \vee \perp \hat{R}_{< 2} \neg p)). \end{aligned}$$

The current-position alternatives in the last line are the derived-operator cases of Eqs. (1) and (3).

Step 2: The two distinct primitive timed subformulas receive clocks x and y , respectively. Their clocked-LTL translation is

$$\begin{aligned} \Phi_U &= p \vee (\top \wedge X(((x \leq 5) \wedge \text{unch}(x) \wedge \top) \cup \\ &\quad ((x \leq 5) \wedge p))), \\ \Phi_R &= \neg p \wedge (\perp \vee (\text{rst}(y) \wedge \\ &\quad X(\neg p \vee ((y \geq 2) \vee (\perp \wedge \neg p))))) , \\ \mathcal{T}(f) &= \Phi_U \wedge \Phi_R. \end{aligned}$$

The upper-bounded Until uses x as a preservation clock. The lower test $y \geq 2$ is the complement of the strict source bound < 2 ; the Release clause therefore permits p beginning exactly at age two.

Step 3: A Spot backend output for $\mathcal{T}(f)$ is shown on the left of Fig. 4. Its cube labels illustrate that clock guards, reset markers, preservation markers, and action atoms are independent propositions at this stage. In particular, cubes containing both p and $\neg p$ are propositionally satisfiable to the backend. The diagram is an illustrative output, not a state-size or performance result.

Step 4: Relaxation removes negative literals over extended clock atoms, and reset completion assigns resets according to the clock class: x is reset unless the transition carries $\text{unch}(x)$, while y is reset exactly when it carries $\text{rst}(y)$. The right side of Fig. 4 shows the resulting TBA transitions after the markers are removed. This is the boundary of the translation theorem; subsequent optimization removes the unsatisfiable action cubes and simplifies the automaton.

5 CORRECTNESS AND CLOCK BOUND

An extended word ρ adds, at every position i , a value $\text{val}_\rho(i, x) \in \mathbb{R}_{\geq 0}$ and reset decision $\text{reset}_\rho(i, x)$ for each $x \in \text{Clock}(f)$. It is clock-consistent when, for $\Delta_i = t_{i+1} - t_i$,

$$\text{val}_\rho(i+1, x) = \begin{cases} \Delta_i, & \text{reset}_\rho(i, x), \\ \text{val}_\rho(i, x) + \Delta_i, & \text{otherwise.} \end{cases} \quad (9)$$

The value at position i is the value tested by guards before the reset decision at that position. Let $\text{same_base}(\rho, w)$ mean that w is the timed-word projection of ρ .

Theorem 5.1 (Correctness of the clock encoding). *For every well-formed formula f and timed word w ,*

$$w, 0 \models_{\text{MTL}} f \iff \exists \rho. \text{same_base}(\rho, w) \wedge \text{clock_consistent}(\rho) \wedge \rho, 0 \models_{\text{LTL}} \mathcal{T}(f).$$

The existential quantifier over ρ is taken jointly over all distinct primitive timed subformulas. Soundness does not rely on choosing the canonical trace: each clock may follow any reset pattern allowed by the encoding, provided its value recurrence is respected. Completeness is the more delicate half because all activations that share a formula-keyed clock must be served simultaneously. The proof constructs the clock traces in parallel, using the residual predicates only for the preservation families and the direct restart rule for the remaining families. This result is the Coq theorem `EncodingCorrect_proved`. Its soundness direction holds for any clock-consistent reset trace. Completeness constructs one canonical trace for all clocks together. The detailed induction, including the strict-bound arithmetic, is in Supplement 2.

Corollary 5.2 (One clock per distinct primitive timed subformula). *All occurrences of the same primitive timed subformula use the same logical clock. The encoding uses exactly the clocks in $\text{Clock}(f)$, hence at most one clock per distinct primitive timed subformula, independently of the number of syntax-tree occurrences or run-time activations.*

Proof. Every clock in $\mathcal{T}(f)$ is indexed by an element of $\text{Clock}(f)$, and `T_uses_every_clock` proves that every such element occurs in $\mathcal{T}(f)$. A single extension assigns one value and reset trace to each element of $\text{Clock}(f)$. Theorem 5.1 proves that the canonical trace suffices for every pending activation of that formula. $\square$

Define `compile_with(A)` by applying relaxation and reset completion to backend automaton A . Timed acceptance is existential over clock-consistent extensions of the timed

TABLE 4
Coq lemmas for the primitive timed proof cases. Each row denotes both strict and non-strict comparisons.

Primitive family	Soundness	Canonical completeness
$\hat{U}_{\leq d}$	<code>sound_UhatLe</code>	<code>complete_UhatLe</code>
$\hat{U}_{\geq d}$	<code>sound_UhatGe</code>	<code>complete_UhatGe</code>
$\hat{R}_{\leq d}$	<code>sound_RhatLe</code>	<code>complete_RhatLe</code>
$\hat{R}_{\geq d}$	<code>sound_RhatGe</code>	<code>complete_RhatGe</code>
$\hat{U}_{< d}$	<code>sound_UhatLt</code>	<code>complete_UhatLt</code>
$\hat{U}_{> d}$	<code>sound_UhatGt</code>	<code>complete_UhatGt</code>
$\hat{R}_{< d}$	<code>sound_RhatLt</code>	<code>complete_RhatLt</code>
$\hat{R}_{> d}$	<code>sound_RhatGt</code>	<code>complete_RhatGt</code>

word and requires that reset decisions agree with the reset sets along an accepting run.

Theorem 5.3 (MTL-to-TBA correctness). *For every well-formed f , backend automaton A satisfying Assumption 4.1, and timed word w ,*

$$w, 0 \models_{\text{MTL}} f \iff \text{TBA_accepts}(\text{compile_with}(A), w).$$

In Coq this is `MTL_to_TBA_correct_with`; the backend correctness is an explicit hypothesis, not a project-specific axiom.

Proof. Combine Theorem 5.1 with Proposition 4.3. The forward direction uses the canonical extension and reset-completion lemma; the reverse direction uses upward closure under relaxation, the backend contract, and clock-encoding soundness. $\square$

5.1 Proof structure

The Coq proof separates local timed reasoning from structural composition. Boolean, next, and untimed Until/Release cases follow by induction; ordinary timed operators unfold to the hatted primitives. The full case analysis and arithmetic lemmas are in Supplement 2.

Soundness. For any clock-consistent extension, a preserved clock accumulates elapsed time, while a reset clock is no greater than the time elapsed since its most recent reset. Thus the clock guards imply the corresponding elapsed-time bounds, with strict inequalities preserved. Until clauses provide the required endpoint and prefix; Release clauses cover their bounded windows. For lower-bounded Until, divergence justifies the fairness branch when the threshold is never reached. These arguments depend on clock consistency, not on the canonical reset policy.

Completeness. The canonical extension assigns one reset trace to each formula-keyed clock. Residual predicates retain an older reference while it can still satisfy its obligation and cover newer activations; otherwise the policy resets for the current activation. This gives one clock trace for all overlapping activations of the same formula. For lower-bounded Until, if no threshold is reached, the active-obligation invariant yields the fairness branch. Strict cases use strict residuals and the strict increase of timestamps. The supplement gives the primitive-by-primitive induction and its arithmetic details.

The local cases are assembled into the theorem `EncodingCorrect_proved`. Its completeness argument

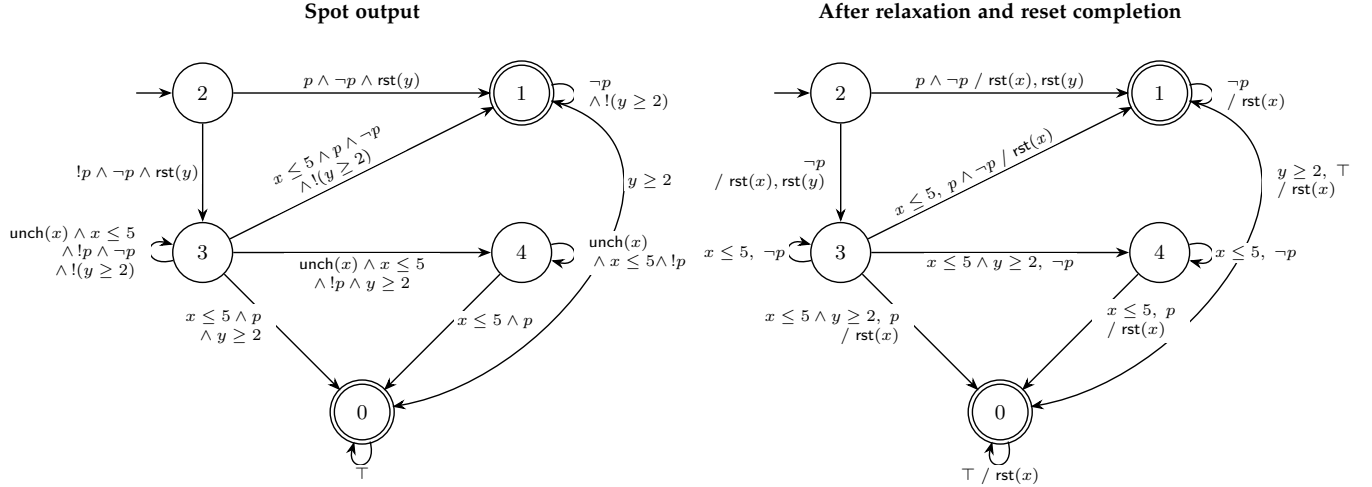


Fig. 4. Spot 2.16 output (TikZ redraw, left; raw DOT and input in `examples/spot/`) and reset-completed TBA (right; same state IDs). Here $p = \text{LAct}(p)$, $\neg p = \text{LNAct}(p)$, and $!p$ is the literal negation of $\text{LAct}(p)$. Timed semantics makes $!p$ and $\neg p$ redundant action exclusions; duplicate exclusions are suppressed, while $p \wedge \neg p$ remains for the later TBA simplification. Edges on the right are labeled “guard, action / resets.”

TABLE 5
Scope of the mechanized translation theorem.

Component	Status
Ordinary-operator definitions and pointwise compatibility	Coq-verified
Eight hatted clocked-LTL clauses	Coq-verified
Canonical reset policy and residual completeness	Coq-verified
Relaxation and reset completion	Coq-verified
LTL-to-Büchi language equivalence	Explicit assumption
Parser, backend process, output printer	Outside theorem

proves equivalence for every node identified by a syntax-tree path, not only for the root. This stronger induction invariant is needed because repeated formula occurrences share the same clock while the induction traverses separate paths.

What the mechanization assumes. The proof boundary is summarized in Table 5. In particular, the correctness of the untimed backend is an explicit proposition-level assumption; the timed proof does not depend on Spot or on determinism. In the included development, the theorem `MTL_to_TBA_correct_with` accepts that contract as an argument. The parser, backend invocation and output printer are not part of this theorem.

The detailed arithmetic lemmas and primitive soundness and completeness cases are in Supplement 3; the full reset-policy and composition arguments are in Supplement 1.

6 RELATED WORK

MTL and its dense-time pointwise semantics have a substantial literature, including decidability and expressiveness results [1], [2], [4]–[6]. The one-sided-bounded fragment is a useful compromise: unrestricted pointwise dense-time MTL does not admit a general effective translation of all formulas to finite ordinary timed automata. Results under bounded-variability assumptions or with alternating timed automata have different scopes [9], [10]; $\text{MTL}_{0,\infty}$ retains common

TABLE 6
Qualitative comparison with CASAAL. The table summarizes the cited construction and tool against the result proved in this paper.

Aspect	Comparison
Automaton construction	CASAAL: Dedicated tableau-style construction for timed obligations. This paper: Clocked-LTL encoding, propositional Büchi backend, relaxation, and reset completion.
Timed references	CASAAL: Timed reasoning is integrated into its specialized construction. This paper: Hatted operators preserve the triggering timestamp while logical evaluation moves to the next event.
Proof boundary	CASAAL: Correctness is established for the cited specialized construction. This paper: Coq proves the timed encoding and reset completion under an explicit correctness contract for the LTL-to-Büchi backend.
Output use	CASAAL: Exact nondeterministic automata support translation; approximations are available when determinism is required for controller synthesis. This paper: Exact, possibly nondeterministic property automata; the theorem does not require determinism.
Clock sharing	CASAAL: Handled by the specialized timed construction. This paper: One clock per distinct primitive timed subformula; residual obligations and the canonical reset policy prove completeness under repeated activations.

response and separation requirements [2], [3]. The hatted operators in this paper make the distinction between a timing reference and the current logical position explicit. They are primitive constructors in the logic and proof, so their strict pointwise behavior is not delegated to an unverified preprocessing pass.

Translations of timed logics. The closest complete automata construction for $\text{MTL}_{0,\infty}$ is CASAAL, which uses a dedicated tableau-style method and derives deterministic approximations when controller synthesis requires them [7], [8]. Bulychev et al. study controller synthesis for the same fragment [3]. Our objective is complementary: the formula is compiled to an exact nondeterministic TBA, which can

then be combined with a system model. For MITL, classical constructions use temporal testers, networks of automata, or one-clock alternating automata [10]–[12]. Tools such as MightyL and MITL2Timed provide practical translations and UPPAAL-oriented workflows [13], [14]; they target a different logic and architecture. Existing techniques for merging timed obligations are related to our clock-reuse problem, but the residual predicates here are tailored to one-sided bounds and are part of the machine-checked completeness proof.

Clocked temporal logic and subformula clocks. CLTL_{oc} extends LTL with clock comparisons and reset predicates and reduces satisfiability to a theory solver; it has also been used to encode metric logics [15]. Our clocked-LTL formula uses a similar propositional vocabulary but serves as an intermediate representation for automaton synthesis. The backend treats clock tests and reset markers as propositions; positive clock atoms make the language upward closed, while preservation markers and reset completion recover their timed meaning. This relationship motivates the explicit relaxation proof rather than assuming that the backend understands clocks.

One clock per subformula is also familiar from event-clock logics and subformula-based translations [9], [16], [17]. Those clocks typically record time since, or until, an event. Here a clock is keyed by a timed formula and may face multiple overlapping activations of that same formula. The canonical trace resolves those competing references; its correctness is proved for each strict and non-strict primitive.

Recent automata-based real-time verification. Bouyer, Srivathsan, and Vishwanath improve an MTL-to-one-clock alternating timed automaton construction for finite timed words using explicit clock deactivation, identify a bounded-width fragment, and develop zone-based emptiness checking [18]. Their alternating-automaton and zone-analysis setting differs from our infinite-word TBA target, but is closely related in its treatment of clock activity. Wang et al. formally verify a transformation from finite, discrete-time MLTL formulas to regular expressions [19]; this is adjacent mechanized translation work with a different time domain and output language. TEMPORA provides pointwise MITL model checking with past and future modalities using generalized timed automata and zone-based algorithms [20]. It is an adjacent tool pipeline rather than an equivalent MTL_{0,∞}-to-TBA translation. Ho et al. present MightyPPL, a compositional tool for pointwise MITL with past and Pnueli modalities that generates timed automata for finite and infinite timed words [21]. Its construction merges overlapping obligations and bounds the number of identical component testers. This is a practical model-checking pipeline for a different logic and automaton architecture; our result instead proves a formula-keyed clock bound for MTL_{0,∞} and the stated event-guarded TBA semantics.

Mechanized translations and trusted backends. Mechanized work has verified LTL-to-Büchi translations and model checkers [22]–[25]. For metric logics, PVS developments formalize corrected decision procedures for MITL [26]; Coq has also been used for quantitative MTL monitoring [27]. These are complementary targets: our theorem concerns infinite dense-time words and the formula-

keyed clocks required by an MTL-to-TBA construction. The theorem deliberately assumes the correctness of the LTL-to-Büchi backend. A verified LTL translator could discharge that contract if its output were adapted to the automaton interface used by the Coq development. Our result thus isolates the timed translation proof from the backend rather than claiming that Spot itself is verified.

Table 6 summarizes how this modular proof boundary differs from CASAAL, the closest complete construction for the same one-sided-bounded fragment. The comparison concerns proof architecture and intended use; it does not claim a general automaton-size advantage. The clock-sharing row states the guarantee proved here; it does not imply that CASAAL allocates one clock per syntax-tree occurrence or uses more clocks.

7 CONCLUSION

We have presented a mechanically verified semantic translation from MTL_{0,∞} to event-guarded timed Büchi automata without location invariants and with existential initial clock valuations. Hatted operators make the timing reference explicit; the clocked-LTL clauses cover strict and non-strict bounds; and relaxation with reset completion turns a correct propositional Büchi backend into a timed automaton. The canonical reset policy and residual obligations establish that one clock per distinct primitive timed subformula suffices even with arbitrarily many overlapping activations. Coq proves both semantic preservation and the end-to-end result `MTL_to_TBA_correct_with` under the stated backend contract.

The result is a proof-backed translation layer for real-time verification tools, rather than a standalone finite-input compiler. A practical pipeline still needs finite-bound encoding, backend integration, and any required initialization and export adapters. The Coq development, proof sources, and build instructions for this paper are available in the project repository at <https://github.com/efares1/coq-verified-mtl-to-tba>.

REFERENCES

- [1] R. Koymans, “Specifying real-time properties with metric temporal logic,” *Real Time Syst.*, vol. 2, no. 4, pp. 255–299, 1990. [Online]. Available: <https://doi.org/10.1007/BF01995674>
- [2] R. Alur, T. Feder, and T. A. Henzinger, “The benefits of relaxing punctuality,” *Journal of the ACM (JACM)*, vol. 43, no. 1, pp. 116–146, 1996.
- [3] P. Bulychev, A. David, K. Larsen, and G. Li, “Efficient controller synthesis for a fragment of MTL(0,∞),” *Acta Informatica*, vol. 51, no. 3–4, pp. 165–192, Jan. 2014.
- [4] D. D’Souza and P. Prabhakar, “On the expressiveness of MTL in the pointwise and continuous semantics,” *International Journal on Software Tools for Technology Transfer*, vol. 9, no. 1, pp. 1–4, 2007.
- [5] J. Ouaknine and J. Worrell, “On the decidability of metric temporal logic,” in *20th Annual IEEE Symposium on Logic in Computer Science (LICS 2005)*. IEEE Computer Society, 2005, pp. 188–197.
- [6] P. Bouyer, F. Chevalier, and N. Markey, “On the expressiveness of TPTL and MTL,” in *Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2005)*, ser. Lecture Notes in Computer Science, vol. 3821. Springer, 2005, pp. 432–443.
- [7] G. Li, P. G. Jensen, K. G. Larsen, A. Legay, and D. B. Poulsen, “Practical controller synthesis for MTL_{0,∞},” in *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software*, ser. SPIN 2017. New York, NY, USA: Association for Computing Machinery, 2017, pp. 102–111. [Online]. Available: <https://doi.org/10.1145/3092282.3092303>

- [8] G. Li, “CASAAL: A tool for translating $MTL_{0,\infty}$ into timed Büchi automata,” Online, 2017, available at the CASAAL tool page. [Online]. Available: <https://lcs.ios.ac.cn/~ligy/tools/CASAAL/>
- [9] D. Ničković and N. Piterman, “From MTL to deterministic timed automata,” in *Formal Modeling and Analysis of Timed Systems (FORMATS 2010)*, ser. Lecture Notes in Computer Science, vol. 6246. Springer, 2010, pp. 152–167.
- [10] T. Brihaye, M. Estiévenart, and G. Geeraerts, “On MITL and alternating timed automata over infinite words,” 2014. [Online]. Available: <https://arxiv.org/abs/1406.4395>
- [11] O. Maler, D. Nickovic, and A. Pnueli, “From MITL to timed automata,” in *Formal Modeling and Analysis of Timed Systems (FORMATS 2006)*, ser. Lecture Notes in Computer Science, vol. 4202. Springer, 2006.
- [12] T. Ferrère, O. Maler, D. Ničković, and A. Pnueli, “From real-time logic to timed automata,” *Journal of the ACM*, vol. 66, no. 3, 2019, art. no. 19.
- [13] T. Brihaye, G. Geeraerts, H.-M. Ho, and B. Monmege, “MightyL: A compositional translation from MITL to timed automata,” in *Computer Aided Verification*, R. Majumdar and V. Kunčák, Eds. Cham: Springer International Publishing, 2017, pp. 421–440.
- [14] “MITL2Timed: Modifying Oddoux and Gastin LTL2BA to timed automata generation,” GitHub repository. [Online]. Available: <https://github.com/yzh89/MITL2Timed>
- [15] M. M. Bersani, M. Rossi, and P. San Pietro, “A tool for deciding the satisfiability of continuous-time metric temporal logic,” *Acta Informatica*, vol. 53, no. 2, pp. 171–206, 2016.
- [16] R. Alur, L. Fix, and T. A. Henzinger, “Event-clock automata: A determinizable class of timed automata,” *Theoretical Computer Science*, vol. 211, no. 1–2, pp. 253–273, 1999.
- [17] J.-F. Raskin and P.-Y. Schobbens, “The logic of event clocks: Decidability, complexity and expressiveness,” *Journal of Automata, Languages and Combinatorics*, vol. 4, no. 3, pp. 247–286, 1999.
- [18] P. Bouyer, B. Srivathsan, and V. Vishwanath, “Model-checking real-time systems: Revisiting the alternating automaton route,” in *Foundations of Software Science and Computation Structures (FoSSaCS 2025)*, ser. Lecture Notes in Computer Science, vol. 15691. Springer Nature Switzerland, 2025, pp. 399–421.
- [19] Z. Wang, K. Kosaian, and K. Y. Rozier, “Formally verifying a transformation from MLTL formulas to regular expressions,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2025)*, ser. Lecture Notes in Computer Science, vol. 15696. Springer, 2025, pp. 254–275.
- [20] S. Akshay, P. Contractor, P. Gastin, R. Govind, and B. Srivathsan, “TEMPORA: Efficient verification of metric temporal properties with past in pointwise semantics,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 640–659.
- [21] H.-M. Ho, S. N. Krishna, K. Madnani, R. Majumdar, and P. Pandya, “MightyPPL: Model checking MITL with past and Pnueli modalities,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 457–479.
- [22] A. Schimpf, S. Merz, and J.-G. Smaus, “Construction of Büchi automata for LTL model checking verified in Isabelle/HOL,” in *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, ser. Lecture Notes in Computer Science, vol. 5674. Springer, 2009, pp. 424–439.
- [23] J. Esparza, P. Lammich, R. Neumann, T. Nipkow, A. Schimpf, and J.-G. Smaus, “A fully verified executable LTL model checker,” in *Computer Aided Verification (CAV 2013)*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 463–478.
- [24] A. Schimpf and P. Lammich, “Converting linear-time temporal logic to generalized Büchi automata,” *Archive of Formal Proofs*, 2014, https://isa-afp.org/entries/LTL_to_GBA.html, formal proof development.
- [25] B. Seidl and S. Sickert, “A compositional and unified translation of LTL into ω -automata,” *Archive of Formal Proofs*, https://isa-afp.org/entries/LTL_Master_Theorem.html, 2019.
- [26] N. Roohi and M. Viswanathan, “Revisiting MITL to fix decision procedures,” in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2018)*, ser. Lecture Notes in Computer Science, vol. 10747. Springer, 2018, pp. 474–494.
- [27] A. Chattopadhyay and K. Mamouras, “A verified online monitor for metric temporal logic with quantitative semantics,” in *Runtime Verification (RV 2020)*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 383–403.